

# Conda Cheat Sheet

## Environment Management

Create environment with Python version

```
conda create -n myenv python=3.11
```

Activate environment

```
conda activate myenv
```

Deactivate current environment

```
conda deactivate
```

List all environments

```
conda env list
```

Clone an existing environment

```
conda create -n newenv --clone oldenv
```

Delete an environment

```
conda env remove -n myenv
```

Show details of active environment

```
conda info --envs
```

**Never work in base.** Always create a dedicated environment per project — if something breaks, delete only that environment without affecting anything else.

## Installing Packages

Install a single package

```
conda install numpy
```

Install multiple packages at once

```
conda install numpy pandas matplotlib
```

Install a specific version

```
conda install numpy=1.24
```

Install minimum version

```
conda install "numpy>=1.20"
```

Install from conda-forge channel

```
conda install -c conda-forge package
```

Install with pip (last resort)

```
pip install some-package
```

Install pip package inside conda env

```
conda run -n myenv pip install pkg
```

**Channel priority:** defaults → conda-forge → pip. Prefer conda; mixing pip can cause conflicts.

## Managing Packages

List installed packages

```
conda list
```

Search for a package

```
conda search numpy
```

Check if a package is installed

```
conda list numpy
```

Update a specific package

```
conda update numpy
```

Update all packages in environment

```
conda update --all
```

Update conda itself

```
conda update -n base conda
```

Remove a package

```
conda remove numpy
```

Dry-run install (check for conflicts)

```
conda install --dry-run numpy
```

Pin a package to prevent updates

```
echo "numpy==1.24" >> $CONDA_PREFIX/conda-meta/pinned
```

Show config file locations

```
conda config --show-sources
```

## Sharing & Reproducing

Export environment to file

```
conda env export > environment.yml
```

Export without build strings (portable)

```
conda env export --no-builds > environment.yml
```

Export only manually installed packages

```
conda env export --from-history >
```

```
environment.yml
```

Recreate environment from file

```
conda env create -f environment.yml
```

Update env from modified yml file

```
conda env update -f environment.yml
```

**Always commit** environment.yml to your project repo. Use **--from-history** for the cleanest, most portable export — only packages you explicitly installed are listed.

## Jupyter / VSCode Setup

Register env as a Jupyter kernel:

```
conda activate myenv
conda install ipykernel
python -m ipykernel install \
  --user --name myenv \
  --display-name "Python (myenv)"
```

List registered kernels

```
jupyter kernelspec list
```

Remove a kernel

```
jupyter kernelspec remove myenv
```

In VSCode: open any .ipynb → click kernel picker (top-right) → select **Python (myenv)**. Each project environment should have its own registered kernel.

## Cleanup & Maintenance

Remove all unused packages and cache

```
conda clean --all
```

Remove only cached tarballs

```
conda clean --tarballs
```

Remove index cache

```
conda clean --index-cache
```

Remove unused packages only

```
conda clean --packages
```

Show conda system info

```
conda info
```

Show all config values

```
conda config --show
```

Show config file locations

```
conda config --show-sources
```

Disable auto-activation of base

```
conda config --set auto_activate_base false
```

Add conda-forge as default channel

```
conda config --add channels conda-forge
```

Run **conda clean -all** every few weeks. The package cache accumulates silently and can reach several GB over time.

## When to Create a New Environment

- **Every new project** — always, no exceptions
- **Conflicting library version** — e.g. TF 1.x vs 2.x
- **Collaborating** — recreate from environment.yml
- **Deep learning** — PyTorch & TensorFlow need isolated CUDA
- **Reproducing a paper** — pin exact package versions
- **Testing a major upgrade** — upgrade safely without breaking main env

## Essential Data Science Packages

| Package      | Purpose                    |
|--------------|----------------------------|
| numpy        | Arrays & linear algebra    |
| pandas       | DataFrames, data wrangling |
| matplotlib   | Plotting                   |
| seaborn      | Statistical visualisation  |
| scikit-learn | Classical ML algorithms    |
| xgboost      | Gradient boosting          |
| jupyter      | Notebook interface         |
| ipykernel    | VSCode kernel bridge       |
| pytorch      | Deep learning (GPU ready)  |
| tensorflow   | Deep learning (Google)     |
| scipy        | Scientific computing       |
| statsmodels  | Statistical models         |

```
conda create -n projectname python=3.11
conda activate projectname
conda install numpy pandas matplotlib seaborn scikit-learn jupyter ipykernel
python -m ipykernel install -user -name projectname -display-name "Python (projectname)"
```